

MONITOR Z · L01

Anatomía de un programa Assembler: el prólogo de ZTKMONS

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L01 — Anatomía de un programa

Assembler: el prólogo de ZTKMONS

Objetivo: entender línea por línea **cómo entra y cómo sale** un programa de z/OS, cómo recibe el `PARM` y por qué ZTKMONS es `AMODE 31 RMODE 24`. Todo lo que viene después (el bucle, el SSI, el socket) vive entre este prólogo y este epílogo.

Usamos el código real: `plataforma_cierres\vivo\host\ZTKMONS.asm`. Los números de línea son de ese archivo.

1. Cómo llega el control a tu programa

Cuando el sistema (o otro programa) te llama, **los registros traen información por convención:**

Registro	Qué trae al entrar	Para qué lo usás
R1	Dirección de la lista de parámetros	Leer el <code>PARM</code> del <code>EXEC</code>
R13	Dirección de la save area del que te llamó (18 palabras = 72 bytes)	Guardar ahí sus registros
R14	Dirección de retorno	Volver al terminar
R15	Tu dirección de entrada	Usarla como base

Y al salir **vos** tenés que dejar:

Registro	Qué dejás al salir
R0–R14	Los mismos valores que te dieron (los restaurás)
R15	El return code . Es el <code>COND CODE</code> que ves en la bitácora

 Fuente: *z/OS MVS Programming: Assembler Services Guide*, capítulo *Linkage conventions* (convención estándar con save area de 72 bytes).

🌟 Analogía: la posta del relevo

Cuando te pasan la posta, el que te la da te deja cuatro cosas en la mano: **R1** = la mochila con el encargo (el PARM), **R13** = su libreta, donde tenés que anotar cómo te dejó las cosas, **R14** = el lugar adonde tenés que volver, y **R15** = tu propio número de corredor. Al volver le devolvés todo como estaba, más **una nota en R15**: cómo te fue (el RC).

Al ENTRAR (te lo dan)

R1	Dirección de la lista de parámetros (el PARM)
R13	Dirección de la save area del que te llamó (72 bytes)
R14	Dirección de retorno
R15	Tu dirección de entrada → sirve de base

Al SALIR (lo dejás vos)

R0-R14	Igual a como te los dieron (los restaurás)
R15	El return code : es el COND CODE que ves en la bitácora (0000, 0004, 0008...)

Si al salir R15 vale 8, en el JESMSGGLG ves \$HASP395 ... ENDED - RC=0008 .

Fuente: z/OS MVS Programming: Assembler Services Guide, capítulo "Linkage conventions"

Lámina L01-01_registros_al_entrar_y_salir

2. La save area: la "pila" del mundo z/OS

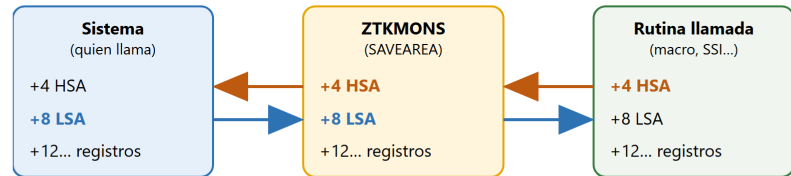
z/OS no usa una pila de hardware. Cada programa tiene un bloque de 18 palabras y los bloques **se encadenan**:

Offset	Contenido
0	(uso de lenguajes, ignorado)
4	HSA : puntero a la save area <i>anterior</i> (la de quien te llamó)
8	LSA : puntero a la save area <i>siguiente</i> (la de a quien llamás)
12	R14
16	R15
20 ... 68	R0, R1, ..., R12

Es una lista doblemente enlazada. Gracias a ella, un dump puede reconstruir quién llamó a quién.

Offset	Contenido (18 palabras = 72 bytes)
0	uso de lenguajes (se ignora)
4	HSA → save area anterior
8	LSA → save area siguiente
12	R14
16	R15
20	R0
24	R1
28 ... 64	R2 ... R11
68	R12

Se encadenan como vagones (lista doble):



azul = LSA (hacia adelante: a quién llamé)

naranja = HSA (hacia atrás: quién me llamó)

Gracias a esta cadena, un dump reconstruye quién llamó a quién, aunque no haya una pila de hardware.

🚂 Analogía: los vagones de un tren

Cada programa engancha su vagón (su save area) al del que lo llamó: un gancho mira hacia atrás (HSA) y otro hacia adelante (LSA). Para volver, cada uno se suelta y vuelve al vagón de atrás.

Fuente: z/OS MVS Programming: Assembler Services Guide, "Linkage conventions" (formato de la save area de 72 bytes)

Lámina L01-02_save_area

3. El prólogo de ZTKMONS (líneas 89–101)

```

ZTKMONS  CSECT
          STM  R14,R12,12(R13)    (1) guardo R14..R12 en la save area
          LR   R12,R15             (2) R12 = mi dirección de entrada →
          LA   R11,4095(,R12)      (3) R11 = R12 + 4095 ...
          LA   R11,1(,R11)         ... + 1 = R12 + 4096 → segunda base
          LA   R10,4095(,R11)      (4) R10 = R11 + 4096 → tercera base
          LA   R10,1(,R10)
          USING ZTKMONS,R12,R11,R10 (5) le aviso al ensamblador cuáles se
          ST   R1,SALVAR1          (6) guardo el puntero al PARM para de
          LA   R2,SAVEAREA         (7) R2 = MI save area
          ST   R13,4(,R2)          (8) mi HSA = la del que me llamó
          ST   R2,8(,R13)          (9) su LSA = la mía
          LR   R13,R2              (10) desde ahora R13 apunta a MI save
    
```

Por qué cada paso:

- **(1)** `STM R14,R12,12(R13)` guarda 15 registros (R14, R15, R0 ... R12; el rango "da la vuelta") a partir del offset 12. **No toca R13**: ese lo guardamos aparte en (8).
- **(2)–(5) Registros base.** Una instrucción RX/RS direcciona con base + desplazamiento, y el desplazamiento tiene **12 bits (0–4095)**. Por eso un

registro base cubre 4 KB de código y datos. ZTKMONS es grande, así que usa tres bases contiguas (R12, R11, R10): unos 12 KB direccionables. El `LA ...,4095 + LA ...,1` existe porque `LA` no admite un desplazamiento de 4096.

- **USING** no genera código: es una **promesa al ensamblador**: "en ejecución, R12 va a valer la dirección de ZTKMONS". Si la promesa es falsa, el programa direcciona basura.
- **(6)** R1 se pisa enseguida (macros como OPEN y WTO lo usan), por eso se guarda.
- **(7)–(10)** Encadenamiento. Después de (10) ya podés llamar a otros (macros del sistema, SSI) sin romper la save area del que te llamó.

 Fuente: *HLASM Language Reference* (instrucción USING y direccionamiento base-desplazamiento) · *z/Architecture Principles of Operation* (formatos RX/RS, desplazamiento de 12 bits).

El prólogo de ZTKMONS, paso a paso (líneas 89–101)

L01 · Anatomía de un programa Assembler

ZTKMONS CSECT

STM R14,R12,12(R13)

LR R12,R15

LA R11,4095(,R12)

LA R11,1(,R11)

LA R10,4095(,R11)

LA R10,1(,R10)

USING ZTKMONS,R12,R11,R10

ST R1,SALVAR1

LA R2,SAVEAREA

ST R13,4(,R2)

ST R2,8(,R13)

LR R13,R2

(1) guardo R14..R12 en la save area DEL QUE ME LLAMÓ

(2) R12 = mi dirección de entrada → 1.ª base

(3) R11 = R12 + 4095 ...

... + 1 = R12 + 4096 → 2.ª base

(4) R10 = R11 + 4096 → 3.ª base

(5) le aviso al ensamblador cuáles son mis bases

(6) guardo el puntero al PARM para después

(7) R2 = MI save area

(8) mi HSA = la del que me llamó

(9) su LSA = la mía

(10) desde ahora R13 apunta a MI save area

Paso	Qué logra	Por qué
(1)	No romperle nada al que me llamó	Guarda 15 registros (R14, R15, R0...R12: el rango "da la vuelta") desde el offset 12. R13 no : va aparte en (8)
(2)–(4)	Tres registros base contiguos	Cada base cubre 4 KB. <code>LA</code> no admite desplazamiento 4096, por eso 4095 + 1
(5)	Que el ensamblador sepa direccionar	<code>USING</code> no genera código : es una promesa sobre lo que valdrán R12, R11 y R10
(6)	No perder el PARM	OPEN, WTO y otras macros pisan R1
(7)–(10)	Enganchar mi vagón a la cadena	Desde acá ya puedo llamar a macros y al SSI sin romper la save area del que me llamó

Fuente: vivo\host\ZTKMONS.asm, líneas 89–101 · HLASM Language Reference (USING)

Lámina L01-03_prologo_paso_a_paso

Memoria del programa (desde el inicio de ZTKMONS) →

R12 cubre +0 ... +4095 (4 KB)	R11 cubre +4096 ... +8191 (R12 + 4096)	R10 cubre +8192 ... +12287 (R11 + 4096)	fuera de las bases
---	--	---	------------------------------

Una instrucción RX/RS direcciona con **base + desplazamiento**, y el desplazamiento tiene **12 bits: 0 a 4095**.

Ejemplo: un dato a +5000 se direcciona con R11 y desplazamiento $5000 - 4096 = 904$.

Las tablas grandes (NUEVA, VIEJA, REPORT) van al final, **fuera** de las bases, y se usan por dirección =A(...).

🔦 Analogía: linternas en un pasillo oscuro

Cada registro base es una linterna que ilumina **exactamente 4 metros** (4096 bytes) de pasillo. ZTKMONS mide unos 12 metros de código y datos, así que necesita **tres linternas** puestas una a continuación de la otra. Lo que queda más allá (las tablas grandes) no se ilumina con linterna: se va con la dirección exacta en la mano.

Si la promesa del USING es falsa (por ejemplo, olvidás cargar R11), el ensamblador no se queja, pero en ejecución el programa lee y escribe en lugares equivocados.

Fuente: z/Architecture Principles of Operation (formatos RX/RS, desplazamiento de 12 bits) · ZTKMONS.asm

Lámina L01-04_registros_base

4. El epílogo (líneas 220–224)

SALIR	DS	0H	
	L	R13,4(,R13)	vuelvo a la save area del que me llamo
	L	R14,12(,R13)	recupero su dirección de retorno
	LM	R0,R12,20(R13)	recupero R0..R12
	BR	R14	vuelvo. R15 = return code (NO se resta)

Detalle fino: el epílogo restaura R0–R12 desde el offset 20, **no** R14–R12 desde el offset 12. Si hiciera `LM R14,R12,12(R13)`, pisaría R15 con el valor que traía al entrar, y **el RC se perdería**. Muchos programas usan la macro `RETURN (14,12),RC=(15)`, que hace exactamente esto.

✓ Lo que hace ZTKMONS (líneas 220–224)

```
SALIR DS 0H
L R13,4(,R13)    vuelvo al vagón de atrás
L R14,12(,R13)   mi dirección de retorno
LM R0,R12,20(R13) R0..R12
BR R14           R15 = RC, intacto
```

Restaura todo **menos R15**, que lleva el RC.

✗ El error clásico

```
L R13,4(,R13)
LM R14,R12,12(R13) ← también R15
BR R14
```

Pisa R15 con el valor que traía al **entrar** (la dirección de entrada), y el RC calculado **se pierde**.

Forma	Qué hace con R15
LM R0,R12,20(R13) + L R14,12(,R13)	Lo deja como está → el RC llega a JES
RETURN (14,12),RC=(15)	Lo mismo, en forma de macro
LM R14,R12,12(R13)	Lo pisa con lo que traía al entrar

📦 Analogía: devolver la caja de herramientas

Devolvés cada herramienta a su lugar, pero la **nota con el resultado del trabajo** (R15) la dejás arriba. Si guardás también la nota vieja encima, el dueño lee la nota de ayer.

Fuente: vivo\host\ZTKMONS.asm, líneas 220–224 · Assembler Services Guide (RETURN)

Lámina L01-05_epilogo

5. Cómo llega el PARM (rutina PARSEA, línea 773)

Con `//PASO EXEC PGM=ZTKMONS,PARM='ZTKD* 5 20'`, el sistema arma esto:



```
L R2,SALVAR1      R2 = R1 original (la lista)
L R2,0(,R2)       R2 = dirección del área del PARM
LH R3,0(,R2)      R3 = longitud (media palabra)
LTR R3,R3
BNP PMALO         sin PARM → error
...
LA R2,2(,R2)      R2 = primer carácter del texto
```

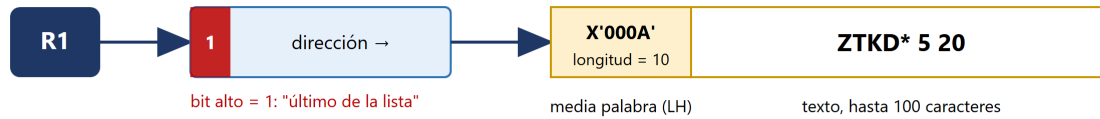
¿Y el bit alto? La palabra trae el bit 0 encendido (marca de fin de lista). En **AMODE 31** la dirección efectiva usa sólo los 31 bits de la derecha, así que el bit se ignora al direccionar. En **AMODE 24** también se ignora (usa 24 bits). **No**

sería inocuo si comparás o sumás esa palabra como número: ahí habría que limpiarlo con `N R2,=X'7FFFFFFF'`.

Cómo llega el PARM (rutina PARSEA, línea 773)

L01 · Anatomía de un programa Assembler

//PASO EXEC PGM=ZTKMONS,PARM='ZTKD* 5 20'



La 1.ª palabra de la lista trae el bit 0 encendido. En AMODE 31 la dirección efectiva usa los 31 bits de la derecha, así que al **direccionar** el bit se ignora. Si la usás como **número** (comparar, sumar), límpiala: `N R2,=X'7FFFFFFF'`.

L	R2,SALVAR1	R2 = R1 original (la lista)
L	R2,0(R2)	R2 = dirección del área del PARM
LH	R3,0(R2)	R3 = longitud (media palabra)
LTR	R3,R3	
BNP	PMALO	sin PARM (longitud 0) → error
LA	R2,2(R2)	R2 = primer carácter del texto

📌 Analogía: el sobre con la carta

R1 te da la dirección de un **casillero**; en el casillero hay un **sobre**; el sobre dice en el frente cuántas hojas trae (la longitud) y adentro está la carta (el texto). El sello rojo (bit alto) sólo dice "es el último sobre del casillero".

Fuente: vivo\host\ZTKMONS.asm, rutina PARSEA (línea 773) · Assembler Services Guide (lista de parámetros del EXEC PARM)

Lámina L01-06_como_llega_el_parm

6. AMODE 31, RMODE 24 y el DCB (líneas 1–3 y 933–935)

- **AMODE** (*addressing mode*) dice con cuántos bits direcciona el programa al ejecutar: 24 bits = 16 MB, 31 bits = 2 GB.
- **RMODE** (*residence mode*) dice **dónde puede cargarse**. RMODE 24 = debajo de la "línea" de 16 MB.
- ZTKMONS es **RMODE 24** porque su **DCB de QSAM** (`PRTDCB`, para `SYSPRINT`) vive dentro de la CSECT, y QSAM exige el DCB bajo la línea. Lo dice el comentario de la línea 933.
- Es **AMODE 31** porque las áreas que devuelve JES por SSI 80 pueden estar **arriba** de la línea.

Después del OPEN, el programa verifica que abrió:


```

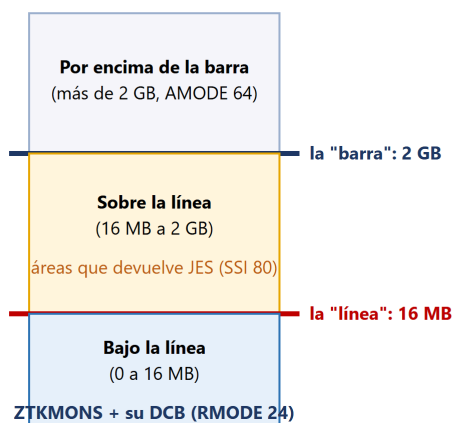
OPEN  (PRTDCB,OUTPUT)
TM    PRTDCB+48,X'10'      DCBOFLGS (offset 48), bit DCBOFOPN =
BO    CONPRT
WTO   'ZTKMONS: FALTA EL DD SYSPRINT'

```

Fuente: z/OS MVS Programming: Assembler Services Guide (AMODE/RMODE) · z/OS DFSMS Macro Instructions for Data Sets (DCB, campo DCBOFLGS y bit de OPEN exitoso).

AMODE, RMODE y por qué el DCB vive "abajo"

L01 · Anatomía de un programa Assembler



	Qué significa	En ZTKMONS
AMODE	Con cuántos bits direcciona al ejecutar: 24 → 16 MB · 31 → 2 GB · 64 → arriba de la barra	31 : las áreas de JES pueden estar sobre la línea
RMODE	Dónde puede cargarse el programa: 24 = bajo la línea; ANY = donde sea	24 : el DCB está dentro de la CSECT
DCB de QSAM	Describe el archivo (SYSPRINT). QSAM exige que esté bajo la línea	PRTDCB, línea 934
¿Abrió?	Después del OPEN se mira DCBOFLGS (offset 48), bit X'10'	TM PRTDCB+48,X'10'

⚡ Analogía: un edificio con un ascensor viejo

La memoria es un edificio. QSAM es un empleado antiguo que **no sube escaleras**: sólo atiende en la **planta baja** (bajo la línea). Por eso el DCB, que es el formulario que él tiene que leer, debe estar en planta baja (RMODE 24). En cambio, el programa puede **visitar** cualquier piso hasta el 2 GB (AMODE 31), porque JES le deja las respuestas en pisos altos.

Fuente: Assembler Services Guide (AMODE/RMODE) · z/OS DFSMS Macro Instructions for Data Sets (DCB, DCBOFLGS) · ZTKMONS.asm líneas 1–3 y 933–935

Lámina L01-07_amode_rmode

7. Dos decisiones de diseño que vamos a revisar más adelante

Decisión en v0	Por qué está bien hoy	Cuándo cambia
La save area y los datos están dentro de la CSECT (SAVEAREA DS 18F, línea 930) → el programa no es reentrante	Un job que corre una sola copia no lo necesita	L07: un STC con varias tareas o en LPA debería ser reentrante (GETMAIN del área de trabajo)

Las tablas grandes (NUEVA, VIEJA, REPORT) van al final , fuera de las tres bases , y se usan por dirección =A(...)	Si estuvieran en medio, harían falta más registros base	Si el programa crece más de 12 KB de código, falta otra base o hay que partirlo
--	---	---

Dos decisiones de v0 que vamos a revisar

L01 · Anatomía de un programa Assembler

Decisión en ZTKMONS v0	Por qué está bien hoy	Cuándo cambia
Save area y datos dentro de la CSECT (SAVEAREA DS 18F, línea 930) → no es reentrante	Un job que corre una sola copia no lo necesita	L07: un STC con varias tareas o cargado en LPA debería ser reentrante (área de trabajo con GETMAIN / STORAGE)
Tablas grandes (NUEVA, VIEJA, REPORT) al final, fuera de las bases, usadas por =A(...)	Si estuvieran en medio, harían falta más registros base	Si el código pasa los 12 KB, falta otra base o hay que partir el programa

Analogía: el formulario pegado en la pared

Un programa **no reentrante** es un formulario pegado en la pared donde cada uno escribe encima: sirve si lo usa una sola persona. Uno **reentrante** es un formulario modelo del que cada uno saca **su propia fotocopia** (su área de trabajo) para escribir.

Fuente: vivo\host\ZTKMONS.asm, líneas 925-944

Lámina L01-08_decisiones_de_diseno

8. Preguntas de comprobación

1. Si te olvidás de la línea (9) (ST R2,8(R13)), ¿el programa falla? ¿Qué es lo que sí se pierde?
2. ¿Por qué el epílogo usa LM R0,R12,20(R13) y no LM R14,R12,12(R13)? ¿Qué vería la bitácora si se usara el segundo?
3. Una instrucción está a 9.000 bytes del inicio de ZTKMONS. ¿Con qué registro base la direcciona el ensamblador y con qué desplazamiento?
4. Si corrés ZTKMONS **sin** PARM, ¿qué valor tiene R3 después del LH y por qué rama sale?
5. ¿Qué pasaría si ZTKMONS fuera RMODE ANY?

9. Práctica en DESA (la escribís vos)

1. Abra SYSPRINT (LRECL 133) y verifique el OPEN con el **TM** de la sección 6.
2. Escriba una línea **+HOLA PARM=<texto del PARM>**.
3. Termine con **RC = 4** si vino PARM y **RC = 8** si no vino.

- el JOBID de cada corrida;
- el RC que da \$HASP395 ... ENDED - RC= en el JESMSG LG (tiene que dar 0004 y 0008);
- el SYSPRINT.

Si el ensamblado da errores, traé el listado de ASMA90: **leer esos errores es parte de la lección**. Lo archivamos en `20 - Practicas` como `P01 - ZTKHOLA`.

Pista: para escribir una línea, mirá cómo lo hace la rutina `MANDA` de `ZTKMONS ( PUT PRTDCB, SALIDA )`.

Práctica P01: ZTKHOLA (la escribís vos)

L01 · Anatomía de un programa Assembler

Paso	Qué tiene que hacer el programa	Qué reusás de ZTKMONS
1	Mismo prólogo : STM, base, USING, guardar R1 y encadenar la save area	Líneas 89–101
2	Abrir SYSPRINT (LRECL 133) y verificar el OPEN	<code>OPEN + TM ...+48,X'10'</code>
3	Escribir <code>+HOLA PARM=<texto del PARM></code>	Rutina <code>MANDA (PUT PRTDCB, SALIDA)</code> y <code>PARSEA</code>
4	<code>RC = 4</code> si vino PARM y <code>8</code> si no vino	<code>LA R15,...</code> antes del epílogo
5	Mismo epílogo , sin pisar R15	Líneas 220–224

Corrida	PARM	Resultado esperado	Qué traés
A	<code>PARM='PRUEBA'</code>	<code>\$HASP395 ... RC=0004</code> y <code>+HOLA PARM=PRUEBA</code>	JOBID, la línea <code>\$HASP395</code> y el SYSPRINT. Si falla el ensamblado, el listado de ASMA90
B	(sin PARM)	<code>\$HASP395 ... RC=0008</code>	

Leer los errores de ASMA90 **es parte de la lección**: si algo no ensambla, mejor.

Fuente: Lección L01, sección 9

Lámina L01-09_practica_ztkhola

Relacionado: [L00 - Que es un demonio en z-OS y que vamos a construir](#) · [Glosario](#) · [Ruta de aprendizaje](#) · siguiente: [L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB](#).

 Fuente: código `vivo\host\ZTKMONS.asm` (v0, 944 líneas) y los manuales de IBM citados en cada sección.

